Project Report: Job Shop Scheduling Solver

Submitted by: RAO ABDULREHMAN
Course Project — Evolutionary Computing

1. Introduction

For this project, I built an interactive web application that solves the **Job Shop Scheduling Problem (JSSP)**. To find good schedules quickly, I implemented a search algorithm called **Particle Swarm Optimization (PSO)** in JavaScript.

The Job Shop Scheduling Problem is a classic optimization problem. We have a set of jobs that must be processed on a set of machines. Because the number of possible schedules grows extremely fast as you add more jobs and machines, finding the absolute best schedule is very difficult (it is an NP-hard problem).

2. Problem Description

In this scheduling problem, we are given:

- A set of jobs.
- A set of machines.

Each job has a specific sequence of operations that must be processed in order. For example, Job 1 might need to go to Machine 3 first, and then Machine 1. Each operation has a specific processing time (duration).

We must follow two main constraints:

1. **Precedence constraint:** An operation for a job can only start after the previous operation for that same job has completely finished.
2. **Machine constraint:** Each machine can only work on one operation at a time. Once a machine starts an operation, it cannot be interrupted.

The goal is to arrange the operations so that the total time to finish all jobs (called the **makespan**) is minimized.

3. Particle Swarm Optimization (PSO) Algorithm

PSO is an optimization algorithm that is inspired by the way flocks of birds or schools of fish search for food.

Instead of birds, the algorithm uses a group of "particles" that fly through the search space. Each particle represents a candidate scheduling solution. As they move, they remember the best solution they have personally found so far (called `pBest`), and they also know the best solution found by any particle in the entire group (called `gBest`).

In each step of the algorithm, the velocity and position of each particle are updated using these formulas:

```
v = w * v + c1 * r1 * (pBest - x) + c2 * r2 * (gBest - x)
x = x + v
```

Where:

- `v` is the particle's velocity.
- `x` is the particle's current position.
- `w` is the inertia weight, which starts at 0.729 and slowly decreases to 0.4. This helps the swarm explore widely at the start and focus on fine-tuning at the end.
- `c1` and `c2` are acceleration coefficients, both set to 1.494.
- `r1` and `r2` are random numbers between 0 and 1.

4. The Ranked Order Value (ROV) Decoding Rule

Standard PSO works with continuous numbers (decimals), but scheduling requires discrete orders of jobs (like Job 1, then Job 3, then Job 2). To bridge this gap, I used the **Ranked Order Value (ROV)** rule.

First, we represent a schedule sequence by repeating the job numbers. For example, if we have Job 0 and Job 1, and each has 2 operations, our reference sequence is `[0, 0, 1, 1]`.

Then, the particle's continuous position values are sorted to find their ranks. We use these ranks to rearrange our reference sequence. Here is a simple example:

- Reference sequence: `[0, 0, 1, 1]`
- Particle position vector: `[1.2, -3.4, 0.5, 4.1]`
- Sorted values: `-3.4` (index 1), `0.5` (index 2), `1.2` (index 0), `4.1` (index 3)

- Sorted indices: `[1, 2, 0, 3]`
- Rearranged sequence: `[0, 1, 0, 1]` (Job 0, then Job 1, then Job 0, then Job 1)

This method always produces a valid schedule sequence without breaking any precedence constraints.

5. Application Architecture & Code

The project is split into three frontend files and a simple server runner:

- **index.html:** Sets up the layout of the web page. The left sidebar lets you adjust settings like swarm size and iterations, select a preset, or write jobs manually. The right side shows the Gantt chart and results.
- **styles.css:** Contains the styling. I designed it with a modern dark theme and custom layout rules to keep it organized. The Gantt chart bars slide into place using CSS keyframe animations.
- **app.js:** Handles the application logic. It processes the manual inputs and CSV file uploads, runs the PSO algorithm loop, updates the progress bar, and renders the charts.
- **server.js:** A simple Node.js HTTP server script to host the static files locally on port 3000.

6. Testing and Results

I tested the solver using standard presets in the application, including the classic **FT06 benchmark** (a standard test case with 6 jobs and 6 machines).

Using the default settings:

- Swarm Size: 80 particles
- Iterations: 300

The algorithm runs in less than 0.1 seconds in the browser and consistently converges to the optimal makespan of **55**.

Once the algorithm finishes, the app displays a Gantt chart showing when each job runs on each machine, a line chart showing the makespan improving over iterations, and a detailed schedule table.

7. Conclusion

This project was a great way to learn how metaheuristics can solve difficult real-world logistics and manufacturing problems. The PSO algorithm combined with the ROV rule works very fast and provides a highly visual way to see scheduling optimization in action.